Big M Database Design: ERD and Normalisation

Data Management | Team project (3 students) | Logical database design

Overview. Big M is a retail chain with stores across Australia. Each store has departments and employees, sells products, and takes orders from customers in store and online. The task was to turn the written business rules into a logical data model: an Entity Relationship Diagram (ERD) in Crow's Foot notation, with every entity checked against the normal forms.

Approach

1. **Identify entities** from the business rules: store, department, employee, product, customer and order.
2. **Define attributes and keys** for each entity, with a primary key for every entity and foreign keys for every relationship.
3. **Resolve many-to-many relationships** with junction entities: a department can appear in many stores, and an order can contain many products.
4. **Model recursive relationships**: an employee supervises other employees, and a store can supervise other stores.
5. **Set connectivity and participation** (one or many, optional or mandatory) for each relationship, recording an assumption wherever the business rules were silent.
6. **Validate with normalisation**, checking each entity for partial and transitive dependencies.

Assumptions

- Each department appears in at least one store.
- Each employee works in only one department.
- A customer's details can be stored in the system without the customer currently having an order.
- An order must contain at least one product.

Entity Relationship Diagram

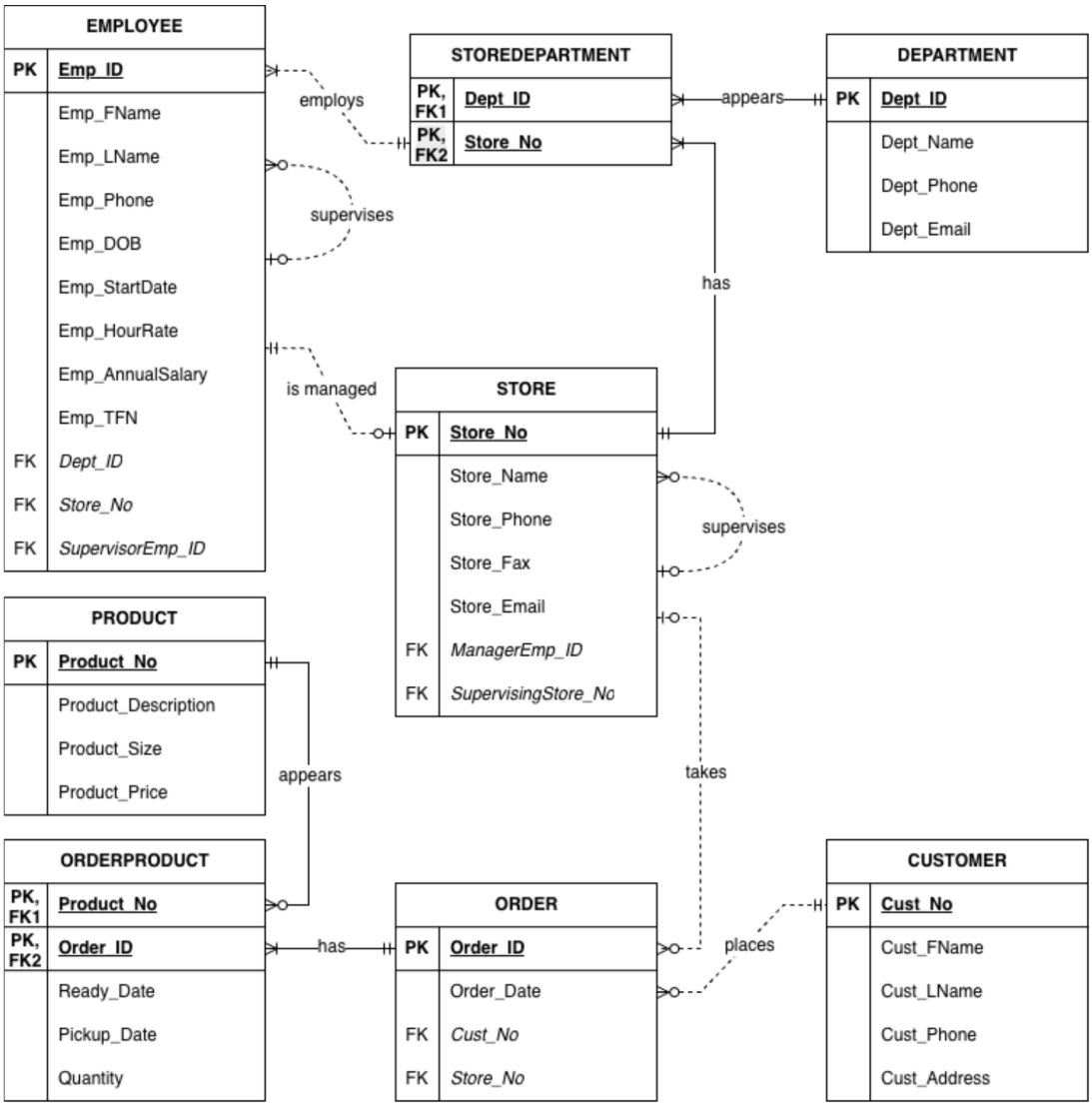


Figure 1: Logical ERD in Crow's Foot notation (primary keys underlined, foreign keys in italics)

Key design decisions

Relationship	Type	How it is modelled
Store and Department	Many-to-many	Junction entity STOREDEPARTMENT, with the composite key (Dept_ID, Store_No). A department can appear in many stores, and a store has many departments.
Store Department and Employee	One-to-many ("employs")	Each employee works in one department of one store, so EMPLOYEE carries Dept_ID and Store_No as foreign keys.
Employee and Employee	Recursive one-to-many ("supervises")	SupervisorEmp_ID in EMPLOYEE points to another employee. The department supervisor supervises the staff in that department.
Employee and Store	One-to-many ("is managed")	ManagerEmp_ID in STORE identifies the employee who manages the store.
Store and Store	Recursive one-to-many ("supervises")	SupervisingStore_No in STORE points to the store that hosts training, payroll, servers and the help desk.

Relationship	Type	How it is modelled
Customer and Order; Store and Order	One-to-many ("places", "takes")	ORDER carries Cust_No and Store_No as foreign keys, so every order is linked to the customer who placed it and the store that took it.
Order and Product	Many-to-many	Junction entity ORDERPRODUCT, with the composite key (Order_ID, Product_No). Ready_Date, Pickup_Date and Quantity sit here, because the dates can differ for each product within the same order.

Normalisation

Every entity was checked for partial dependencies (a non-key attribute depending on only part of a composite key) and transitive dependencies (a non-key attribute depending on another non-key attribute). All eight entities are in **3NF**, so no entity needed to be kept in 2NF.

Entity	Primary key	Normal form	Reason
STORE	Store_No	3NF	Single-attribute key, so no partial dependencies. Store_Name, Store_Phone, Store_Fax, Store_Email, SupervisingStore_No and ManagerEmp_ID depend only on Store_No.
DEPARTMENT	Dept_ID	3NF	Dept_Name, Dept_Phone and Dept_Email depend only on Dept_ID.
STOREDEPARTMENT	(Store_No, Dept_ID)	3NF	Junction entity that resolves the many-to-many link between Store and Department. It holds only the composite key, so there is nothing to depend partially.
EMPLOYEE	Emp_ID	3NF	All attributes, including Dept_ID, Store_No and SupervisorEmp_ID, depend only on Emp_ID.
PRODUCT	Product_No	3NF	Product_Description, Product_Size and Product_Price depend only on Product_No.
CUSTOMER	Cust_No	3NF	Cust_FName, Cust_LName, Cust_Phone and Cust_Address depend only on Cust_No.
ORDER	Order_ID	3NF	Cust_No, Store_No and Order_Date depend only on Order_ID.
ORDERPRODUCT	(Order_ID, Product_No)	3NF	Quantity, Ready_Date and Pickup_Date depend on the whole composite key (the specific product within the specific order), not on either part alone.

Skills demonstrated

Requirements analysis, entity relationship modelling (Crow's Foot notation), primary and foreign keys, connectivity and participation, recursive relationships, resolving many-to-many relationships, normalisation to 3NF, documenting assumptions, and teamwork.